DOCUMENTATION

Dictionary & Most Important Procedures
Animal Welfare System

Proyecto 1

Adrián López Rojas – 2022081429
Carlos Esquivel Solano – 2023367372
Valeria Duran Munoz –2020039557

Base de Datos, 2026

1. Geographic Location Tables

Country — Stores countries for geographic location.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Country name

Province — Provinces belonging to a country.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	Province name
IdCountry	NUMBER(8)	FK → Country, NOT NULL	Parent country

Canton — Cantons belonging to a province.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	Canton name
IdProvince	NUMBER(8)	FK → Province, NOT NULL	Parent province

District — Districts belonging to a canton.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	District name
IdCanton	NUMBER(8)	FK → Canton, NOT NULL	Parent canton

2. Person & Role Tables

Person — Registered users of the system (owners, rescuers, adopters, admins).

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
FirstName	VARCHAR2(25)	NOT NULL	Person first name

Column	Data Type	Constraints	Description
LastName	VARCHAR2(25)	NOT NULL	Person last name
Password	VARCHAR2(60)	NOT NULL	BCrypt hashed password
UserName	VARCHAR2(25)	NOT NULL, UNIQUE	Login username
IdDistrict	NUMBER(8)	FK → District, NOT NULL	Person's district

Email — One or more emails per person.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Email	VARCHAR2(25)	NOT NULL, UNIQUE, CHECK (LIKE '%@%.%')	Email address
IdPerson	NUMBER(8)	FK → Person, NOT NULL	Owner person

Phone — One or more phone numbers per person.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Phone	NUMBER(8)	NOT NULL	Phone number (8 digits)
IdPerson	NUMBER(8)	FK → Person, NOT NULL	Owner person

Admin — Persons with administrator privileges.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
IdPerson	NUMBER(8)	FK → Person, NOT NULL, UNIQUE	Linked person

Rescuer — Persons registered as animal rescuers.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
IdPerson	NUMBER(8)	FK → Person, NOT NULL, UNIQUE	Linked person

Adopter — Persons registered as potential adopters.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
IdPerson	NUMBER(8)	FK → Person, NOT NULL, UNIQUE	Linked person

FosterHome — Persons who offer temporary shelter for animals.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
NeedsDonation	VARCHAR2(1)	NOT NULL, CHECK IN ('Y','N')	Whether food donation is needed
IdPerson	NUMBER(8)	FK → Person, NOT NULL, UNIQUE	Linked person

3. Pet Catalog Tables

PetType — Catalog of pet types (Dog, Cat, Rabbit, etc.).

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Type name

PetBreed — Catalog of breeds linked to a pet type.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Breed name
IdType	NUMBER(8)	FK → PetType, NOT NULL	Parent pet type

PetState — Catalog of pet states (Lost, Found, In Adoption, Adopted, etc.).

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	State name

PetSize — Catalog of pet sizes (Small, Medium, Large, Extra Large).

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Size label

PetLevelEnergy — Catalog of pet energy levels.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Energy level label

PetTraining — Catalog of training difficulty levels.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	Training difficulty

PetSeverity — Catalog of rescue severity levels (Critical, Poor Condition, Good Condition).

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Severity label

SpaceRequired — Catalog of space requirements for pets.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	Space type

PetIllness — Catalog of possible pet illnesses.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	Illness name
Description	VARCHAR2(100)		Illness description

Medicine — Catalog of medicines that can be assigned to pets.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	Medicine name
Dose	VARCHAR2(25)		Recommended dose

PetTreatment — Catalog of treatments applicable to pets.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	Treatment name
Description	VARCHAR2(100)		Treatment description

Currency — Catalog of accepted currencies.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Currency name (e.g. Colones, Dollars)

4. Core Entity Tables

Veterinarian — Veterinarians that can be linked to pets.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
FirstName	VARCHAR2(25)	NOT NULL	First name
LastName	VARCHAR2(25)	NOT NULL	Last name
Name	VARCHAR2(50)	NOT NULL	Clinic or practice name
Email	VARCHAR2(25)	CHECK LIKE '%@%.%'	Contact email
Phone	NUMBER(8)		Contact phone
Location	VARCHAR2(100)		Address description
IdDristrict	NUMBER(8)	FK → District	Veterinarian's district

Pet — Core table storing all registered animals.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL	Pet name
Color	VARCHAR2(50)	NOT NULL	Pet color(s)
Age	NUMBER(8)	CHECK >= 0	Age in years
Description	VARCHAR2(100)		Free text description
Chip	VARCHAR2(15)	UNIQUE	Microchip ID (if any)
IdEnergy	NUMBER(8)	FK → PetLevelEnergy	Energy level
IdState	NUMBER(8)	FK → PetState, NOT NULL	Current state
IdType	NUMBER(8)	FK → PetType, NOT NULL	Pet type
IdBreed	NUMBER(8)	FK → PetBreed	Breed
IdDistrict	NUMBER(8)	FK → District	Current location district
IdSpace	NUMBER(8)	FK → SpaceRequired	Space requirement
IdPetTraining	NUMBER(8)	FK → PetTraining	Training difficulty
IdSize	NUMBER(8)	FK → PetSize	Pet size
IdOwner	NUMBER(8)	FK → Person	Current owner
IdVeterinarian	NUMBER(8)	FK → Veterinarian	Assigned veterinarian

PetPhoto — Photos associated with a pet.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Photo	BLOB	NOT NULL	Photo binary data
IdPet	NUMBER(8)	FK → Pet, NOT NULL	Linked pet

5. Operations Tables

LostReport — Reports of lost pets submitted by their owners.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
LostDate	DATE	NOT NULL	Date the pet was lost
Place	VARCHAR2(100)		Description of where it was lost
Description	VARCHAR2(100)		Additional details

Column	Data Type	Constraints	Description
Reward	NUMBER(12,2)	CHECK >= 0	Offered reward amount
State	VARCHAR2(25)	CHECK IN ('Lost','Found','Closed')	Report status
IdPet	NUMBER(8)	FK → Pet, NOT NULL	Lost pet
IdDistrict	NUMBER(8)	FK → District	District where lost
IdCurrency	NUMBER(8)	FK → Currency, NOT NULL	Reward currency

FoundReport — Reports of found animals submitted by the finder.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
FoundDate	DATE	NOT NULL	Date the pet was found
Place	VARCHAR2(100)		Description of where found
Description	VARCHAR2(100)		Additional details
IdPet	NUMBER(8)	FK → Pet, NOT NULL	Found pet
IdDistrict	NUMBER(8)	FK → District	District where found
IdPerson	NUMBER(8)	FK → Person, NOT NULL	Person who found the pet

PetMatch — Automatic matches between lost and found pets.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
SimilarityPercentage	NUMBER(3)	NOT NULL, CHECK BETWEEN 0 AND 100	Match confidence %
MatchDate	DATE	NOT NULL	Date match was generated
IdLostReport	NUMBER(8)	FK → LostReport, NOT NULL	Linked lost report
IdFoundReport	NUMBER(8)	FK → FoundReport, NOT NULL	Linked found report

Adoption — Adoption records linking a pet to an adopter.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
AdoptionDate	DATE		Date adoption was finalized

Column	Data Type	Constraints	Description
AvailableDate	DATE	NOT NULL	Date pet became available
Description	VARCHAR2(100)		Additional notes
State	VARCHAR2(25)	CHECK IN ('In progress','Adopted','Cancelled')	Adoption status
IdPet	NUMBER(8)	FK → Pet, NOT NULL	Pet being adopted
IdAdopter	NUMBER(8)	FK → Person, NOT NULL	Adopting person
IdOwner	NUMBER(8)	FK → Person, NOT NULL	Previous owner

Rescued — Rescue records for animals found in need.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
RescueDate	DATE	NOT NULL	Date of rescue
Place	VARCHAR2(100)		Description of rescue location
Description	VARCHAR2(100)		Situation description
BeforePhoto	BLOB		Photo before rescue
AfterPhoto	BLOB		Photo after recovery
IdPet	NUMBER(8)	FK → Pet, NOT NULL	Rescued pet
IdDistrict	NUMBER(8)	FK → District	Rescue district
IdRescuer	NUMBER(8)	FK → Rescuer, NOT NULL	Rescuer who performed the rescue
IdPetSeverity	NUMBER(8)	FK → PetSeverity, NOT NULL	Severity at time of rescue

Donation — Donations made by users to animal welfare associations.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Amount	NUMBER(12,2)	NOT NULL, CHECK > 0	Donation amount
DonationDate	DATE	NOT NULL	Date of donation
IdPerson	NUMBER(8)	FK → Person, NOT NULL	Donor
IdCurrency	NUMBER(8)	FK → Currency, NOT NULL	Currency used
IdAssociation	NUMBER(8)	FK → Association, NOT NULL	Beneficiary association

Association — Animal welfare associations that receive donations.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Association name
PhoneNumber	NUMBER(8)		Contact phone
BankAccount	VARCHAR2(35)		IBAN or bank account number
Email	VARCHAR2(25)	CHECK LIKE '%@%.%'	Contact email

6. Social & Audit Tables

Calification — Star ratings and notes given to adopters by previous owners.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Stars	NUMBER(8)	NOT NULL, CHECK BETWEEN 1 AND 5	Rating 1 to 5 stars
Note	VARCHAR2(250)		Optional written note
CalificationDate	DATE	NOT NULL	Date of rating
IdPerson	NUMBER(8)	FK → Person, NOT NULL	Person being rated

ReportList — Reports made against persons (feeds into BlockList).

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Description	VARCHAR2(250)		Reason for report
IdPerson	NUMBER(8)	FK → Person, NOT NULL	Person being reported
IdReporter	NUMBER(8)	FK → Person, NOT NULL, CHECK <> IdPerson	Person making the report
ReportDate	DATE	NOT NULL	Date of report

BlockList — Persons blocked from the adoption system.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
BlockDate	DATE	NOT NULL	Date person was blocked
IdPerson	NUMBER(8)	FK → Person, NOT NULL	Blocked person

Bitacora — Audit log of changes to sensitive fields, populated by triggers.

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
TableName	VARCHAR2(25)	NOT NULL	Table where change occurred
FieldName	VARCHAR2(25)	NOT NULL	Column that changed
PreviousValue	VARCHAR2(50)		Value before change
CurrentValue	VARCHAR2(50)		Value after change
ChangeDate	DATE	NOT NULL	Timestamp of change
ChangedBy	NUMBER(8)	NOT NULL	ID of user who made the change

Parameter — System configuration parameters (e.g. match job interval).

Column	Data Type	Constraints	Description
Id	NUMBER(8)	PK, NOT NULL	Unique identifier
Name	VARCHAR2(25)	NOT NULL, UNIQUE	Parameter name
Value	NUMBER(8)	NOT NULL	Parameter value
Description	VARCHAR2(100)		What the parameter controls

7. Relationship Tables (Many-to-Many)

PetXPetIllness — Many-to-many: pets and their illnesses.

Column	Data Type	Constraints	Description
IdPet	NUMBER(8)	PK, FK → Pet	Pet reference
IdPetIllness	NUMBER(8)	PK, FK → PetIllness	Illness reference

PetXMedicine — Many-to-many: pets and their medicines.

Column	Data Type	Constraints	Description
IdPet	NUMBER(8)	PK, FK → Pet	Pet reference

Column	Data Type	Constraints	Description
IdMedicine	NUMBER(8)	PK, FK → Medicine	Medicine reference

PetXPetTreatment — Many-to-many: pets and their treatments.

Column	Data Type	Constraints	Description
IdPet	NUMBER(8)	PK, FK → Pet	Pet reference
IdPetTreatment	NUMBER(8)	PK, FK → PetTreatment	Treatment reference

PetSizeXFosterHome — Accepted pet sizes per foster home.

Column	Data Type	Constraints	Description
IdPetSize	NUMBER(8)	PK, FK → PetSize	Size reference
IdFosterHome	NUMBER(8)	PK, FK → FosterHome	Foster home reference

PetLevelEnergyXFosterHome — Accepted energy levels per foster home.

Column	Data Type	Constraints	Description
IdPetLevelEnergy	NUMBER(8)	PK, FK → PetLevelEnergy	Energy level reference
IdFosterHome	NUMBER(8)	PK, FK → FosterHome	Foster home reference

SpaceRequiredXFosterHome — Accepted space types per foster home.

Column	Data Type	Constraints	Description
IdSpaceRequired	NUMBER(8)	PK, FK → SpaceRequired	Space type reference
IdFosterHome	NUMBER(8)	PK, FK → FosterHome	Foster home reference

5 Most Important Procedures

This part describes the five most important stored procedures and functions of the Animal Welfare. Each entry includes its purpose, parameter table, annotated source code, and technical notes.

1. SP_GET_PETS_BY_STATE (inside PKG_PET_OPERATIONS)

PROCEDURE

Purpose

Core search procedure for the pet consultation module. Queries the VW_TABLE_ADOPTION view and applies up to 13 optional filters dynamically. Called internally by SP_GET_PETS_UP_FOR_ADOPTION and SP_GET_FOUND_PETS, making it the backbone of all pet search functionality in the system.

Why this procedure matters

This is the most reused procedure in the system. Every pet query — for adoption, found pets, user pets — routes through here. Its dynamic filter design avoids repeating SQL for every search screen.

Parameters

Parameter	Type	Direction	Description
P_ID_STATE	NUMBER	IN	State ID to filter pets by (e.g. 1=Lost, 2=Found, 3=In Adoption)
P_COLOR	VARCHAR2	IN	Optional color filter — partial match (LIKE)
P_AGE	NUMBER	IN	Optional exact age filter
P_NAME	VARCHAR2	IN	Optional name filter — partial match (LIKE)
P_CHIP	VARCHAR2	IN	Optional chip ID filter — partial match
P_ENERGY	VARCHAR2	IN	Optional energy level name filter
P_TYPE	VARCHAR2	IN	Optional pet type name filter
P_BREED	VARCHAR2	IN	Optional breed name filter
P_DISTRICT	VARCHAR2	IN	Optional district name filter
P_SPACE_REQUIRED	VARCHAR2	IN	Optional space requirement name filter
P_TRAINING	VARCHAR2	IN	Optional training level name filter
P_SIZE	VARCHAR2	IN	Optional size name filter
P_VETERINARIAN	VARCHAR2	IN	Optional veterinarian name filter
P_RESULT	SYS_REFCURSOR	OUT	Cursor with matching pet records from VW_TABLE_ADOPTION

Source code (annotated)

```
CREATE OR REPLACE PROCEDURE SP_GET_PETS_BY_STATE(
    P_ID_STATE       IN NUMBER,
    P_COLOR          IN VARCHAR2 DEFAULT NULL,
    P_AGE            IN NUMBER   DEFAULT NULL,
    P_NAME           IN VARCHAR2 DEFAULT NULL,
    P_CHIP           IN VARCHAR2 DEFAULT NULL,
    P_ENERGY         IN VARCHAR2 DEFAULT NULL,
    P_TYPE           IN VARCHAR2 DEFAULT NULL,
    P_BREED          IN VARCHAR2 DEFAULT NULL,
    P_DISTRICT       IN VARCHAR2 DEFAULT NULL,
    P_SPACE_REQUIRED IN VARCHAR2 DEFAULT NULL,
    P_TRAINING       IN VARCHAR2 DEFAULT NULL,
    P_SIZE           IN VARCHAR2 DEFAULT NULL,
    P_VETERINARIAN   IN VARCHAR2 DEFAULT NULL,
    P_RESULT         OUT SYS_REFCURSOR
) IS
BEGIN
    -- Open a cursor on the pet view, applying all optional filters.
    -- NULL or 'ALL' values are treated as no filter for that field.
    OPEN P_RESULT FOR
        SELECT PetId, Color, Age, Description, PetName, Chip,
               Energy, PetState, PetType, Breed, District,
               SpaceRequired, Training, PetSize, OwnerName, VeterinarianName
        FROM VW_TABLE_ADOPTION
        WHERE IdState = P_ID_STATE
          AND (P_COLOR IS NULL OR UPPER(Color) LIKE '%'||UPPER(TRIM(P_COLOR))||'%')
          AND (P_AGE IS NULL OR Age = P_AGE)
          AND (P_NAME IS NULL OR UPPER(PetName) LIKE '%'||UPPER(TRIM(P_NAME))||'%')
          -- ... (same pattern for remaining filters)
        ORDER BY PetId;
END SP_GET_PETS_BY_STATE;
```

Technical notes

- All filters use NULL-safe conditions: passing NULL = no filter applied.
- String filters use LIKE with wildcards for partial match.
- 'ALL' is also accepted as a no-filter value for combo boxes.
- Called by SP_GET_PETS_UP_FOR_ADOPTION and SP_GET_FOUND_PETS with hardcoded state IDs.

---

2. SP_GET_PET_FOR_EDIT

PROCEDURE

Purpose

Retrieves complete pet data for the edit screen, including the full geographic hierarchy (District → Canton → Province → Country). This allows the UI to pre-populate all dropdowns correctly when the user opens the edit form for a pet they own.

Why this procedure matters

Without this procedure, the edit form would not know which country/province/canton to pre-select — it only stores the district ID. The JOIN chain traverses the full hierarchy in a single query.

Parameters

Parameter	Type	Direction	Description
p_pet_id	NUMBER	IN	ID of the pet to retrieve
p_owner_id	NUMBER	IN	ID of the current user — security check to ensure ownership
p_result	SYS_REFCURSOR	OUT	Cursor with pet data including full geographic hierarchy IDs

Source code (annotated)

```
CREATE OR REPLACE PROCEDURE SP_GET_PET_FOR_EDIT (
    p_pet_id   IN  NUMBER,
    p_owner_id IN  NUMBER,
    p_result   OUT SYS_REFCURSOR
) IS
BEGIN
    -- Retrieve pet with full geographic chain for pre-filling edit form.
    -- Security: WHERE clause includes p_owner_id to prevent unauthorized access.
    OPEN p_result FOR
        SELECT
            p.Id AS IdPet,     p.IdOwner,
            p.Color,           p.Age,
            p.Description,     p.Name AS PetName,
            p.Chip,            p.IdEnergy,
            p.IdType,          p.IdBreed,
            p.IdDistrict,
            ca.Id AS IdCanton,
            pr.Id AS IdProvince,
            co.Id AS IdCountry,
            p.IdSpace,         p.IdPetTraining,
            p.IdSize,          p.IdVeterinarian
        FROM Pet p
        INNER JOIN District d  ON d.Id  = p.IdDistrict
        INNER JOIN Canton   ca ON ca.Id = d.IdCanton
        INNER JOIN Province pr ON pr.Id = ca.IdProvince
        INNER JOIN Country  co ON co.Id = pr.IdCountry
        WHERE p.Id = p_pet_id
          AND p.IdOwner = p_owner_id;  -- ownership check
END SP_GET_PET_FOR_EDIT;
```

Technical notes

- The WHERE clause double-checks ownership — a user cannot edit another user's pet.
- The geographic JOIN chain (District → Canton → Province → Country) resolves the full hierarchy needed to pre-populate cascading dropdowns in the UI.
- Returns empty cursor if pet not found or user is not the owner.

---

3. SP_UPDATE_PET_FOR_OWNER

PROCEDURE

Purpose

Updates all editable fields of a pet record. Includes an ownership security check: only the registered owner of the pet can modify it. Returns the number of affected rows so the Java layer can detect whether the update succeeded or was rejected.

Why this procedure matters

Centralizing the update in Oracle ensures the ownership check is enforced at the database level, not just in Java. If someone bypasses the UI, Oracle still prevents unauthorized modifications.

Parameters

Parameter	Type	Direction	Description
p_pet_id	NUMBER	IN	ID of the pet to update
p_owner_id	NUMBER	IN	ID of the current user — ownership check
p_color	VARCHAR2	IN	New color value
p_age	NUMBER	IN	New age value
p_description	VARCHAR2	IN	New description
p_name	VARCHAR2	IN	New pet name
p_chip	VARCHAR2	IN	New chip ID (nullable)
p_id_energy	NUMBER	IN	New energy level ID
p_id_type	NUMBER	IN	New pet type ID
p_id_breed	NUMBER	IN	New breed ID (nullable)
p_id_district	NUMBER	IN	New district ID
p_id_space	NUMBER	IN	New space requirement ID
p_id_pet_training	NUMBER	IN	New training level ID
p_id_size	NUMBER	IN	New size ID
p_id_veterinarian	NUMBER	IN	New veterinarian ID (nullable)
p_rows_updated	NUMBER	OUT	Number of rows affected — 0 means pet not found or not owned by user

Source code (annotated)

```
CREATE OR REPLACE PROCEDURE SP_UPDATE_PET_FOR_OWNER (
    p_pet_id            IN  NUMBER,
    p_owner_id          IN  NUMBER,
    p_color             IN  VARCHAR2,
    p_age               IN  NUMBER,
    p_description       IN  VARCHAR2,
    p_name              IN  VARCHAR2,
```

```
    p_chip             IN  VARCHAR2,
    p_id_energy        IN  NUMBER,
    p_id_type          IN  NUMBER,
    p_id_breed         IN  NUMBER,
    p_id_district      IN  NUMBER,
    p_id_space         IN  NUMBER,
    p_id_pet_training  IN  NUMBER,
    p_id_size          IN  NUMBER,
    p_id_veterinarian  IN  NUMBER,
    p_rows_updated     OUT NUMBER
) IS
BEGIN
    -- Update all pet fields.
    -- WHERE includes IdOwner to enforce ownership at DB level.
    UPDATE Pet
    SET
        Color = p_color,         Age = p_age,
        Description = p_description, Name = p_name,
        Chip = p_chip,           IdEnergy = p_id_energy,
        IdType = p_id_type,      IdBreed = p_id_breed,
        IdDistrict = p_id_district, IdSpace = p_id_space,
        IdPetTraining = p_id_pet_training,
        IdSize = p_id_size,      IdVeterinarian = p_id_veterinarian
    WHERE Id = p_pet_id
      AND IdOwner = p_owner_id;  -- security: only owner can edit

    -- Return affected row count so Java can detect success/failure.
    p_rows_updated := SQL%ROWCOUNT;
END SP_UPDATE_PET_FOR_OWNER;
```

Technical notes

- SQL%ROWCOUNT returns 0 if no row was updated (pet not found OR owner mismatch).
- Java checks p_rows_updated > 0 to determine success.
- This double-layer security (Java + Oracle) prevents unauthorized edits even if the UI is bypassed.

---

4. pr_report_pet_missing

PROCEDURE

Purpose

Changes a pet's state to 'Lost' (IdState = 3) and automatically cancels any open or pending adoption processes for that pet. This keeps data consistent: a lost pet cannot simultaneously be in an active adoption process.

Why this procedure matters

Without this procedure, a pet could be marked as lost while still appearing as available for adoption in the system. The automatic cancellation of adoptions ensures the system state is always consistent.

Parameters

Parameter	Type	Direction	Description
p_pet_id	Pet.Id%TYPE	IN	ID of the pet to mark as missing
p_owner_id	Pet.IdOwner%TYPE	IN	ID of the owner — prevents other users from marking pets as missing

Source code (annotated)

```
CREATE OR REPLACE PROCEDURE pr_report_pet_missing (
    p_pet_id   IN Pet.Id%TYPE,
    p_owner_id IN Pet.IdOwner%TYPE
) IS
BEGIN
    -- Mark the pet as lost (state 3).
    -- WHERE includes IdOwner to prevent unauthorized reports.
    UPDATE Pet
       SET IdState = 3
     WHERE Id      = p_pet_id
       AND IdOwner = p_owner_id;

    -- If no row was updated, the pet doesn't exist or doesn't belong to this user.
    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20005,
            'Pet not found or does not belong to this user.'
        );
    END IF;

    -- Automatically cancel any open adoptions for this pet.
    -- A lost pet cannot have an active adoption process.
    UPDATE Adoption
       SET State = 'Canceled'
     WHERE IdPet  = p_pet_id
       AND State IN ('In process', 'To be confirmed');

    COMMIT;
END pr_report_pet_missing;
```

Technical notes

- Uses %TYPE anchoring (Pet.Id%TYPE) — if the column type ever changes, the parameter automatically adapts.
- RAISE_APPLICATION_ERROR with code -20005 sends a readable error to the Java layer.
- The second UPDATE cancels ALL open adoptions for the pet in a single statement.
- Single COMMIT at the end ensures both updates are atomic — either both succeed or neither does.

---

5. fn_pet_match_score

FUNCTION

Purpose

Calculates a similarity percentage between a lost pet and a found pet based on five weighted criteria: type (25 pts), breed (25 pts), color (20 pts), district (15 pts), and approximate age (15 pts). Used by the match engine to rank candidate matches before inserting into PetMatch.

Why this procedure matters

Encapsulating the scoring logic in a function keeps the match algorithm reusable and testable. The pr_run_match procedure calls this function to score each candidate pair, making it easy to adjust weights without rewriting the match logic.

Parameters

Parameter	Type	Direction	Description
p_lost_pet_id	NUMBER	IN	ID of the lost pet
p_found_pet_id	NUMBER	IN	ID of the found pet
(return)	NUMBER	RETURN	Similarity score from 0 to 100

Source code (annotated)

```
CREATE OR REPLACE FUNCTION fn_pet_match_score (
    p_lost_pet_id  IN NUMBER,
    p_found_pet_id IN NUMBER
) RETURN NUMBER IS
    v_score       NUMBER := 0;
    v_lost_type   NUMBER;  v_found_type   NUMBER;
    v_lost_breed  NUMBER;  v_found_breed  NUMBER;
    v_lost_color  VARCHAR2(50); v_found_color VARCHAR2(50);
    v_lost_dist   NUMBER;  v_found_dist   NUMBER;
    v_lost_age    NUMBER;  v_found_age    NUMBER;
BEGIN
    -- Load both pets' attributes in a single SELECT each.
    SELECT IdType, IdBreed, Color, IdDistrict, Age
    INTO   v_lost_type, v_lost_breed, v_lost_color, v_lost_dist, v_lost_age
    FROM   Pet WHERE Id = p_lost_pet_id;

    SELECT IdType, IdBreed, Color, IdDistrict, Age
    INTO   v_found_type, v_found_breed, v_found_color, v_found_dist, v_found_age
    FROM   Pet WHERE Id = p_found_pet_id;

    -- Criterion 1: Same type (required) — 25 points.
    IF v_lost_type = v_found_type THEN
        v_score := v_score + 25;
    END IF;

    -- Criterion 2: Same breed — 25 points.
    IF v_lost_breed IS NOT NULL AND v_lost_breed = v_found_breed THEN
        v_score := v_score + 25;
    END IF;

    -- Criterion 3: Similar color (partial match) — 20 points.
    IF UPPER(v_lost_color) LIKE '%' || UPPER(v_found_color) || '%'
    OR UPPER(v_found_color) LIKE '%' || UPPER(v_lost_color) || '%' THEN
        v_score := v_score + 20;
    END IF;

    -- Criterion 4: Same district — 15 points.
    IF v_lost_dist IS NOT NULL AND v_lost_dist = v_found_dist THEN
```

```
        v_score := v_score + 15;
    END IF;

    -- Criterion 5: Age within 2 years — 15 points.
    IF v_lost_age IS NOT NULL AND v_found_age IS NOT NULL
       AND ABS(v_lost_age - v_found_age) <= 2 THEN
        v_score := v_score + 15;
    END IF;

    RETURN v_score;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;  -- pet not found = no match
END fn_pet_match_score;
```

Technical notes

- Maximum possible score is 100 (all 5 criteria match).
- Type match is technically optional in the score, but pr_run_match pre-filters by type before calling this function.
- Color uses LIKE bidirectionally: 'White Brown' matches 'White' and 'Brown' matches 'White Brown'.
- Age tolerance of ±2 years handles uncertainty in estimated ages for found strays.
- Returns 0 if either pet ID is invalid (NO_DATA_FOUND exception).